

QA-спецификация: «Свои люди» (app-under-test/index-local.html)

Объект тестирования: [app-under-test/index-local.html](#) — клиентское SPA без сервера, состояние хранится в `localStorage` (`sl_state`, `sl_session`) либо публикуется как Artifact.

Предварительные факты из кода (важны для сценариев ниже):

- `Escape` нигде не обрабатывается — в файле нет ни одного `keydown` / `keyup` - listener. Модалки закрываются только кнопкой `×`, кнопкой «Отмена»/«Заккрыть» или кликом по подложке (`overlay`).
 - В модалках нет `role="dialog"`, `aria-modal`, `aria-labelledby`, нет установки фокуса при открытии и нет `focus-trap` — фокус свободно уходит на фон.
 - Карточки объявлений (`.card[data-open]`) — это `<div>` с обработчиком только `click`, без `tabindex`, `role="button"` и обработки `Enter` / `Space` → недоступны с клавиатуры.
 - У поля цены есть `min="0" step="1" type="number"` — нативная HTML5-валидация блокирует отправку формы через обычный ввод/стрелки, но не через `paste` произвольного текста, DevTools или программный `submit`.
 - Сессия — это только `{username}` в `localStorage`, без пароля/токена; проверки прав (`isAuthor`, `isBooker`) идут по точному совпадению `username`, полученному после регистронезависимого поиска (`findUser`) при логине/регистрации.
-

1. Функциональность

1.1 Регистрация

№	Сценарий	Шаги	Ожидаемый результат
1.1.1	Успешная регистрация	Открыть вкладку «Регистрация» → заполнить логин (латиница, 3–20 символов), пароль ≥ 4 симв., повтор пароля, имя, телефон → отправить	Аккаунт создаётся, тост «Аккаунт создан», автоматический вход, показывается доска
1.1.2	Логин уже занят	Зарегистрировать <code>ivan82</code> , выйти, повторно зарегистрировать <code>ivan82</code>	Ошибка «Этот логин уже занят», форма не отправляется
1.1.3	Логин с недопустимыми символами	Ввести логин с кириллицей/ пробелом/ спецсимволом или короче 3 / длиннее 20 симв.	Ошибка валидации логина, регистрация не проходит
1.1.4	Пароли не совпадают	Пароль и повтор пароля различаются	Ошибка «Пароли не совпадают»
1.1.5	Короткий пароль	Пароль < 4 символов	Ошибка «Пароль слишком короткий» (или блокировка через <code>minlength</code> до отправки)
1.1.6	Не заполнены имя/телефон	Оставить поле «Ваше имя» или «Телефон» пустым (в т.ч. только пробелы)	Требуется поле / ошибка «Заполните имя и телефон» — форма не сохраняется
1.1.7	Квартира необязательна	Зарегистрироваться без указания квартиры	Регистрация проходит успешно

1.2 Вход

№	Сценарий	Шаги	Ожидаемый результат
1.2.1	Успешный вход	Ввести существующий логин/пароль	Вход выполнен, отображается доска, <code>sl_session</code> содержит канонический <code>username</code>
1.2.2	Неверный пароль	Верный логин, неверный пароль	Ошибка «Неверный пароль»
1.2.3	Несуществующий логин	Ввести логин, которого нет	Ошибка «Такого логина нет...»
1.2.4	Выход	Нажать «Выйти»	Сессия очищается (<code>sl_session</code> удалён), показывается экран входа
1.2.5	Персистентность сессии	Войти → перезагрузить страницу (F5)	Пользователь остаётся авторизован (сессия читается из <code>localStorage</code>)

1.3 Размещение объявления

№	Сценарий	Шаги	Ожидаемый результат
1.3.1	Успешное размещение	Заполнить заголовок, категорию, цену, описание → «Разместить»	Объявление появляется в списке первым (сортировка по дате), автор/ кв./телефон подставлены из профиля
1.3.2	Обязательные поля	Оставить заголовок или описание пустым (в т.ч. только пробелы)	Отправка блокируется (HTML <code>required</code> + JS-проверка после <code>trim()</code>)
1.3.3	Категория по умолчанию	Не менять select	Сохраняется категория «Другое»
1.3.4	Отмена создания	Открыть модальку, нажать «Отмена» или × или кликнуть по подложке	Модалка закрывается, объявление не создаётся

1.4 Бронирование и раскрытие телефона

№	Сценарий	Шаги	Ожидаемый результат
1.4.1	Успешная бронь	Другой пользователь открывает чужое объявление → «Забронировать»	Статус меняется на «Забронировано», в модальке появляется номер телефона автора, кнопки «Позвонить» (<code>tel:</code>) и «Скопировать номер»
1.4.2	Карточка после брони	Вернуться к списку	Карточка показывает бейдж «Забронировано: <имя>»
1.4.3	Копирование номера	Нажать «Скопировать номер»	Номер попадает в буфер обмена (или, при недоступности Clipboard API, показывается тостом)
1.4.4	Ссылка «Позвонить»	Проверить <code>href</code> кнопки	<code>href="tel:<цифры и + из телефона автора>"</code>
1.4.5	Видимость телефона третьим лицам	Забронировать объявление одним пользователем → открыть то же объявление третьим (посторонним) пользователем, не автором и не бронировавшим	Наблюдение из <code>qa-report.md</code> / <code>qa-report-solo.md</code> , подтверждено дважды независимо: телефон автора и кнопки «Позвонить»/«Скопировать номер» видны любому авторизованному пользователю, открывшему забронированное объявление, а не только автору и тому, кто забронировал (<code>renderDetail()</code> показывает <code>phone-reveal</code> всем при <code>item.status === 'booked'</code> , без проверки <code>isAuthor</code> / <code>isBooker</code> для самого блока — эта проверка есть только у кнопки «Отменить бронь» внутри блока). Открытый вопрос к продукту: задумано ли это как публичная информация после брони, или это дефект приватности — решение не принято, задокументировано здесь, чтобы не потерялось

1.5 Права на удаление / отмену брони (по логину)

№	Сценарий	Шаги	Ожидаемый результат
1.5.1	Удаление своего объявления	Автор открывает своё объявление	Видна кнопка «Удалить объявление»; удаление проходит
1.5.2	Нет удаления чужого	Не-автор открывает чужое объявление	Кнопки «Удалить объявление» нет в DOM вовсе (не просто скрыта/задизейблена)
1.5.3	Отмена брони автором	Объявление забронировано другим пользователем; автор открывает его	Кнопка «Отменить бронь» видна автору, отмена работает, статус → «Свободно»
1.5.4	Отмена брони тем, кто бронировал	Пользователь, который забронировал, открывает объявление	Кнопка «Отменить бронь» видна, отмена работает
1.5.5	Нет отмены брони посторонним	Третий пользователь (не автор и не бронировавший) открывает забронированное объявление	Кнопки «Отменить бронь» нет в DOM
1.5.6	Проверка по логину, а не по имени	Два пользователя с разным <code>username</code> , но одинаковым <code>displayName</code> (см. 3.x)	Права считаются строго по <code>username</code> / <code>bookedBy.username</code> , а не по отображаемому имени

2. Доступность

2.1 Клавиатурная навигация

№	Сценарий	Шаги	Ожидаемый результат / известный статус
2.1.1	Табуляция по доске	<code>Tab</code> от начала страницы через поиск → чипы категорий → карточки → кнопки	Все интерактивные элементы должны получать фокус в логичном порядке. Известная проблема: карточки <code>.card[data-open]</code> — это <code><div></code> без <code>tabindex</code> , <code>Tab</code> их пропускает, открыть объявление с клавиатуры нельзя
2.1.2	Активация чипов категорий с клавиатуры	<code>Tab</code> до чипа → <code>Enter</code> / <code>Space</code>	Чип активируется (это настоящие <code><button></code> , должно работать)
2.1.3	Кнопка «+ Разместить объявление» с клавиатуры	<code>Tab</code> → <code>Enter</code>	Модалка открывается
2.1.4	Заполнение форм с клавиатуры	Полностью пройти регистрацию/вход/добавление объявления без мыши	Все поля доступны через <code>Tab</code> , отправка формы через <code>Enter</code>
2.1.5	Видимый фокус	Пройтись <code>Tab</code> по всем интерактивным элементам	На каждом должен быть виден focus-outline (в CSS есть <code>:focus{outline:2px solid var(--accent)}</code> только для <code>.search</code> , <code>.field input/select/textarea</code> — проверить кнопки и чипы, у них нет явного <code>:focus</code> стиля, наследуется браузерный)

2.2 Фокус в модальных

№	Сценарий	Шаги	Ожидаемый результат / известный статус
2.2.1	Фокус при открытии модальной	Открыть модальку добавления объявления или деталей	Дефект: фокус не переводится на модальку (нет <code>.focus()</code> при открытии) — он остаётся на элементе, с которого был клик, либо теряется
2.2.2	Focus trap	Открыть модальку → <code>Tab</code> до последнего элемента → ещё <code>Tab</code>	Дефект: фокус уходит на элементы фонового контента под <code>overlay</code> (нет focus-trap), хотя фон визуально перекрыт
2.2.3	Возврат фокуса после закрытия	Открыть модальку с кнопки → закрыть (×/Escape/клик вне)	Ожидается возврат фокуса на кнопку, которая открыла модальку — сейчас не реализовано, проверить фактическое поведение

2.3 Работа `Escape`

№	Сценарий	Шаги	Ожидаемый результат
2.3.1	<code>Escape</code> закрывает модальку добавления	Открыть модальку «Новое объявление» → нажать <code>Escape</code>	Подтверждённый дефект: ничего не происходит, обработчика нет — модалька остаётся открытой
2.3.2	<code>Escape</code> закрывает модальку деталей/брони	Открыть карточку объявления → нажать <code>Escape</code>	Тот же дефект — не закрывается
2.3.3	<code>Escape</code> не должен ничего ломать на доске	Нажать <code>Escape</code> , когда модальных нет	Ничего не происходит (регресс-проверка, что добавление обработчика в будущем не создаст побочных эффектов)

2.4 axe-core на всех экранах

Прогнать `@axe-core/playwright` (`AxeBuilder`) на каждом из состояний, зафиксировать о нарушениях уровня `critical/serious` или задокументировать каждое как известный дефект:

№	Экран	Как получить состояние
2.4.1	Экран входа (Войти)	Открыть приложение без сессии
2.4.2	Экран регистрации	Переключить вкладку «Регистрация»
2.4.3	Доска объявлений (пусто)	Войти пользователем без объявлений
2.4.4	Доска объявлений (со списком)	Войти пользователем, есть ≥1 объявление
2.4.5	Модалка «Новое объявление»	Открыть через «+ Разместить объявление»
2.4.6	Модалка деталей — своё объявление	Открыть свою карточку
2.4.7	Модалка деталей — чужое, свободно	Открыть чужую незабронированную карточку
2.4.8	Модалка деталей — чужое, забронировано (с раскрытым телефоном)	Открыть чужую забронированную карточку
2.4.9	Toast-уведомление видимо на экране	Вызвать любое действие с тостом («Аккаунт создан» и т.п.) во время сканирования

Отдельно проверить руками (ахе не всегда это ловит): отсутствие

`role="dialog"` / `aria-modal="true"` / `aria-labelledby` у `.modal`, что модалки не объявляются экранным читалкам как диалоговое окно.

3. Устойчивость к чужим/кривым данным

№	Сценарий	Шаги	Ожидаемый результат
3.1	Пустые обязательные поля — регистрация	Отправить форму регистрации с пустыми/пробельными: логин, пароль, имя, телефон (по одному и все сразу)	Ни один пустой обязательный вариант не создаёт пользователя; для пробельных значений срабатывает JS-проверка после <code>trim()</code>
3.2	Пустые обязательные поля — объявление	Отправить форму объявления с пустым/пробельным заголовком и/или описанием	Объявление не создаётся (JS-проверка <code>`!item.title`</code>)
3.3	Отрицательная цена — через UI	В поле «Цена, ₽» ввести <code>-500</code> и попытаться отправить	Поле имеет <code>min="0"</code> : браузер должен заблокировать нативной валидацией. Проверить фактическое поведение в целевых браузерах (Chromium/Firefox/WebKit)
3.4	Отрицательная цена — обход валидации	Через DevTools/консоль изменить значение <code>input[name=price]</code> или напрямую вызвать <code>dispatchEvent(new Event('submit'))</code> на форме, либо снять/обойти атрибут <code>min</code>	Зафиксировать как факт: приложение не имеет серверной/JS-проверки цены (<code>`Number(fd.get('price'))`</code>)
3.5	Нечисловая/дробная/огромная цена	Вставить через paste: <code>abc</code> , <code>1.5</code> , <code>9999999999999999</code>	Проверить фактическое отображение (<code>fmtPrice</code> через <code>toLocaleString('ru-RU')</code>) — не должно ломать вёрстку/вызывать ошибку рендера
3.6	Совпадающий логин в разном регистре	Зарегистрировать <code>Ivan82</code> , выйти, попытаться зарегистрировать <code>ivan82</code> / <code>IVAN82</code>	Регистрация отклоняется («Этот логин уже занят») — <code>findUser</code> сравнивает <code>toLowerCase()</code>
3.7	Вход с другим регистром логина	Зарегистрировать <code>Ivan82</code> , выйти, войти как <code>ivan82</code>	Вход должен пройти успешно, <code>session.username</code> при этом должен стать каноническим <code>Ivan82</code> (как хранится у пользователя), а не введённым

№	Сценарий	Шаги	Ожидаемый результат
			<code>ivan82</code> — проверить, что после этого владение объявлениями (<code>authorUsername === me.username</code>) не ломается
3.8	Бронирование собственного объявления через UI	Автор открывает своё же объявление	Кнопки «Забронировать» нет — вместо неё текст «Это ваше объявление — дождитесь брони соседа»
3.9	Бронирование собственного объявления в обход UI	Через консоль вызвать логику брони напрямую (эмуляция клика по несуществующей кнопке <code>#do-book</code> или прямой вызов <code>save()</code> с <code>bookedBy.username === authorUsername</code>)	Зафиксировать как известное ограничение: серверной/повторной проверки «нельзя бронировать своё» нет, защита только на уровне того, что кнопка не рендерится
3.10	Двойное бронирование гонкой	Открыть одно и то же свободное объявление в двух вкладках под разными пользователями, забронировать почти одновременно	Т.к. нет сервера — побеждает та вкладка, чьё сохранение (<code>publish / localStorage</code>) применилось последним; более ранняя бронь будет молча перезаписана. Задокументировать поведение, не блокиер для локальной демо-версии
3.11	Спецсимволы/HTML-инъекция в текстовых полях	В заголовок/описание/имя вставить <code><script>alert(1)</script></code> , <code>"></code>	Не должно исполняться — весь вывод идёт через <code>esc()</code> (HTML-экранирование), проверить визуально, что теги отображаются как текст, а не выполняются
3.12	Удалённый пользователь как автор	Удалить (программно/из старого состояния) пользователя, чьё объявление осталось	Карточка/детали показывают «Пользователь удалён» вместо падения интерфейса
3.13	Подделка сессии в <code>localStorage</code>	Войти как пользователь А, вручную заменить <code>sl_session</code> на <code>{"username": "<пользователь В>"}</code>	Зафиксировать как известное архитектурное ограничение: приложение не проверяет пароль повторно — авторизация «наступает» под именем В без пароля, т.к. сессия не содержит

№	Сценарий	Шаги	Ожидаемый результат
		через DevTools, перезагрузить	токена/подписи. Не блокирует для локального прототипа, но важно задокументировать как security- заметку

4. Нагрузка / объём данных

Цель: проверить поведение приложения на реалистично большом состоянии, сгенерированном программно (не вручную через UI) и подставленном напрямую в `<script id="app-state" type="application/json">` файла `index-local.html` (или в `localStorage['sl_state']` для локального HTTP-сервера — эквивалентно на старте `init()`, т.к. `loadLocalState()` читает именно оттуда, если нет artifact-капабилити).

№	Сценарий	Шаги	Ожидаемый результат / на что смотреть
4.1	Генерация объёма	Скриптом сгенерировать 50+ пользователей (уникальные <code>username</code> , реалистичные <code>displayName</code> / <code>phone</code> / <code>apt</code> , валидные <code>salt</code> / <code>hash</code>) и 200–500 объявлений (случайные категории из <code>CATS</code> , цены, заголовки/описания разной длины, часть — <code>status: 'booked'</code> со случайным <code>bookedBy</code> из сгенерированных пользователей)	Сгенерированное состояние проходит структуру, ожидаемую кодом (<code>state.users</code> , <code>state.listings</code> с теми же полями, что создаёт форма)
4.2	Подстановка в файл	Вставить сгенерированный JSON в <code>#app-state</code> (для автономного файла) и/или в <code>localStorage.sl_state</code> (для варианта с HTTP-сервером, который читает <code>loadLocalState()</code> при отсутствии artifact-капабилити)	Страница загружается без ошибок в консоли, доска отображает объявления
4.3	Отклик поиска (<code>searchQuery</code>) на большом списке	Замерить время от ввода символа в <code>#search</code> до перерисовки <code>.grid</code> (<code>filteredListings()</code> проходит <code>.filter()</code> + <code>.sort()</code> по всем 200–500 записям при каждом нажатии клавиши, т.к. <code>render()</code> вызывается на каждый <code>input</code> -евент без <code>debounce</code>)	Зафиксировать фактическое время рендера (<code>performance.now()</code> до/после); признак деградации — заметная задержка ввода (UI начинает "тормозить" при вводе), т.к. в коде нет <code>debounce</code> /виртуализации списка
4.4	Отклик переключения категории на большом списке	Кликнуть по чипу категории, замерить время до перерисовки	Аналогично 4.3 — <code>filteredListings()</code> + полная пересборка <code>innerHTML</code> всей <code>.grid</code> при каждом клике
4.5	Общий рендер списка (<code>render()</code> целиком)	Замерить время полного вызова <code>render()</code> (например, через <code>performance.now()</code> вокруг вызова, либо <code>console.time</code>) на состоянии с 500 объявлениями	Зафиксировать миллисекунды; ожидаемая деградация — рендер линейно растёт с размером списка, т.к. <code>cardHtml()</code> строкой

№	Сценарий	Шаги	Ожидаемый результат / на что смотреть
			конкатенируется на каждую запись без виртуализации
4.6	Квота <code>localStorage</code> — стандартный путь сохранения	Довести реальный суммарный объём <code>localStorage</code> origin'a вплотную к его фактическому лимиту (лимит браузерозависим — не ориентироваться на «типичные 5–10 МБ»: в тестовом окружении Chromium фактически обнаружено ~50–52 МБ на origin), затем совершить обычное действие с <code>save()</code> (добавить объявление/забронировать) при почти нулевом остатке места	Проверить, вызывает ли <code>localStorage.setItem</code> в <code>saveLocalState()</code> исключение <code>QuotaExceededError</code> при превышении квоты
4.7	Честность сообщения при превышении квоты	Спровоцировать <code>QuotaExceededError</code> (4.6) и проверить, что реально показывает пользователю <code>save()</code> / <code>saveLocalState()</code>	Важно проверить именно код: <code>saveLocalState()</code> оборачивает <code>localStorage.setItem</code> в <code>try{}catch(e){}</code> без обработки ошибки — исключение молча проглатывается, а <code>save()</code> в ветке "нет artifact-капабилити" всё равно вызывает <code>render()</code> и показывает <code>opts.toast</code> (например, «Объявление размещено»/«Бронь подтверждена»), даже если <code>setItem</code> реально бросил исключение и данные не сохранились. Ожидаемый вывод: тост врёт об успехе — честной ошибки о превышении квоты нет
4.8	Фактическая потеря данных после квоты	После сценария 4.6/4.7 перезагрузить страницу (<code>location.reload()</code>)	Если <code>setItem</code> не удался, но тост показал успех — при перезагрузке добавленного объявления/брони не будет (состояние осталось прежним, откатилось к последнему реально сохранённому), что

№	Сценарий	Шаги	Ожидаемый результат / на что смотреть
			подтверждает несоответствие тоста реальности
4.9	Сортировка «забронировано вниз» на большом списке	В сгенерированном состоянии вперемешку расположить <code>open</code> и <code>booked</code> записи (не сгруппированные заранее) с разными <code>createdAt</code> , открыть доску	В <code>filteredListings()</code> сортировка: сначала по <code>status==='booked'</code> (открытые выше), внутри каждой группы — по <code>createdAt</code> по убыванию (новые выше). Проверить на 200–500 записях, что все <code>open</code> идут раньше всех <code>booked</code> , и что внутри каждой группы порядок по дате не нарушен (типичный риск — нестабильная сортировка/сравнение строк <code>createdAt</code> , если даты сгенерированы не в ISO-формате или с одинаковыми значениями)
4.10	Устойчивость <code>.sort()</code> к одинаковым <code>createdAt</code>	Часть сгенерированных записей намеренно сделать с идентичным <code>createdAt</code>	Проверить, не переставляются ли такие записи местами непредсказуемо между перерисовками (сравнение <code>localeCompare</code> на равных строках возвращает 0 — порядок должен остаться стабильным в рамках возможностей <code>Array.prototype.sort</code> движка, но стоит зафиксировать фактическое поведение)

Известные ограничения архитектуры (для отчёта, не баги в привычном смысле)

- Нет сервера: все проверки прав выполняются в клиентском JS и обходимы через DevTools (см. 3.4, 3.9, 3.13).

- Пароли хешируются SHA-256+соль на клиенте — не полноценная криптозащита, весь код (включая хеши) физически виден в источнике страницы.
- Гонки записи (3.10) разрешаются последним успешным сохранением, без блокировок/транзакций.